

SSO AMB JWT A ASIX.CAT

PR0375/03/02



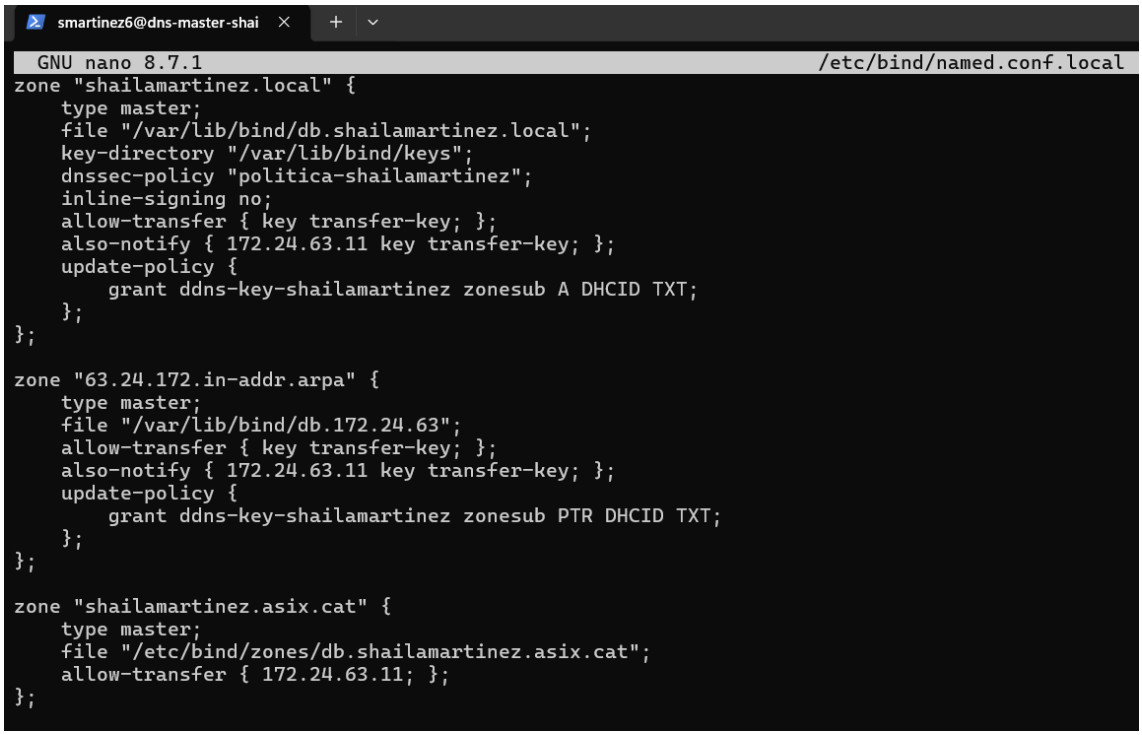
SHAILA MARTÍNEZ
2n ASIX

Índex

Part 1: Zona DNS shailamartinez.asix.cat 2
Part 2: CA pròpia i certificat wildcard 3
Part 3: Claus JWT i serveis d'autenticació i API 5
3.1 Parell de claus RS256 5
3.2 Servei auth 5
3.3 Servei api 6
3.4 Pàgina privada 7
3.5 Docker Compose i usuaris 8
Part 4: Nginx — tres llocs virtuals, TLS i auth_request 9
Part 5: Proves funcionals 11
Part 6: Proves d'atac contra el JWT 15
Qüestions 18

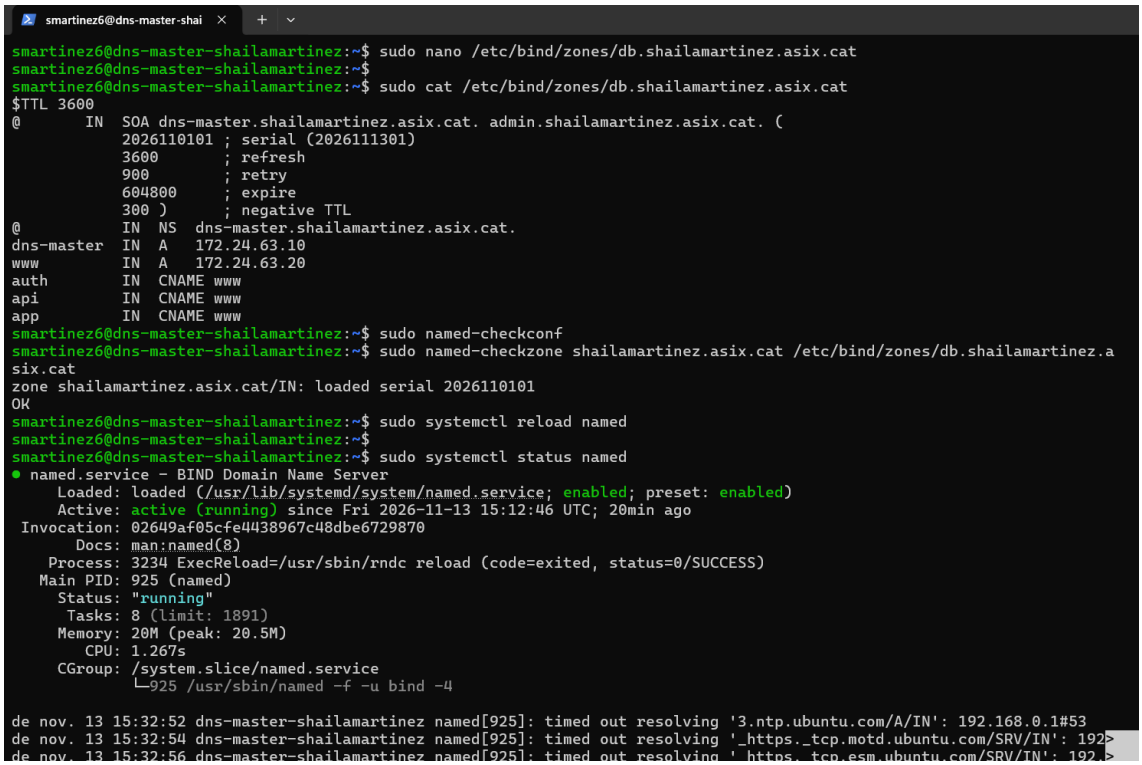
Part 1: Zona DNS shailamartinez.asix.cat

He afegit la nova zona al fitxer named.conf.local del server principal.

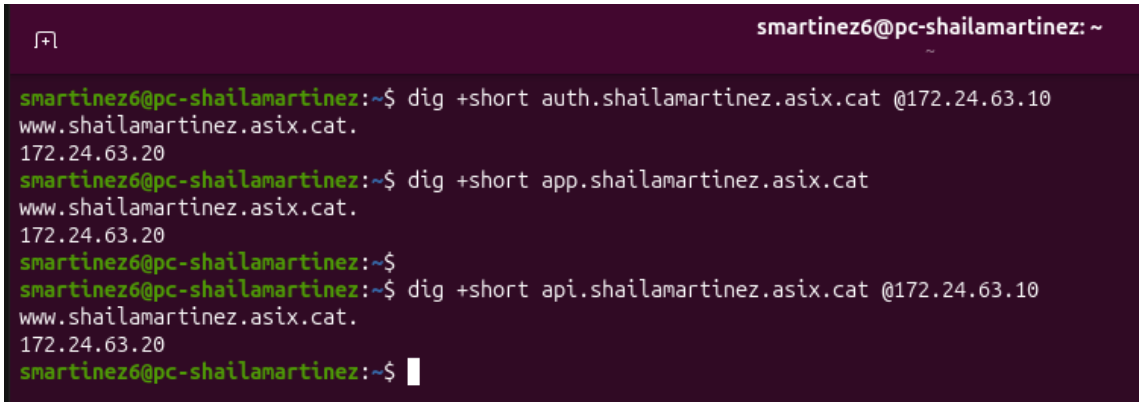


Contingut del fitxer db-shailamartinez.asix.cat. He creat els registres perquè www, auth, api i app apuntin al servidor amb la IP .20

Després comprovo que la configuració no té errors i el resultat de la zona es OK.

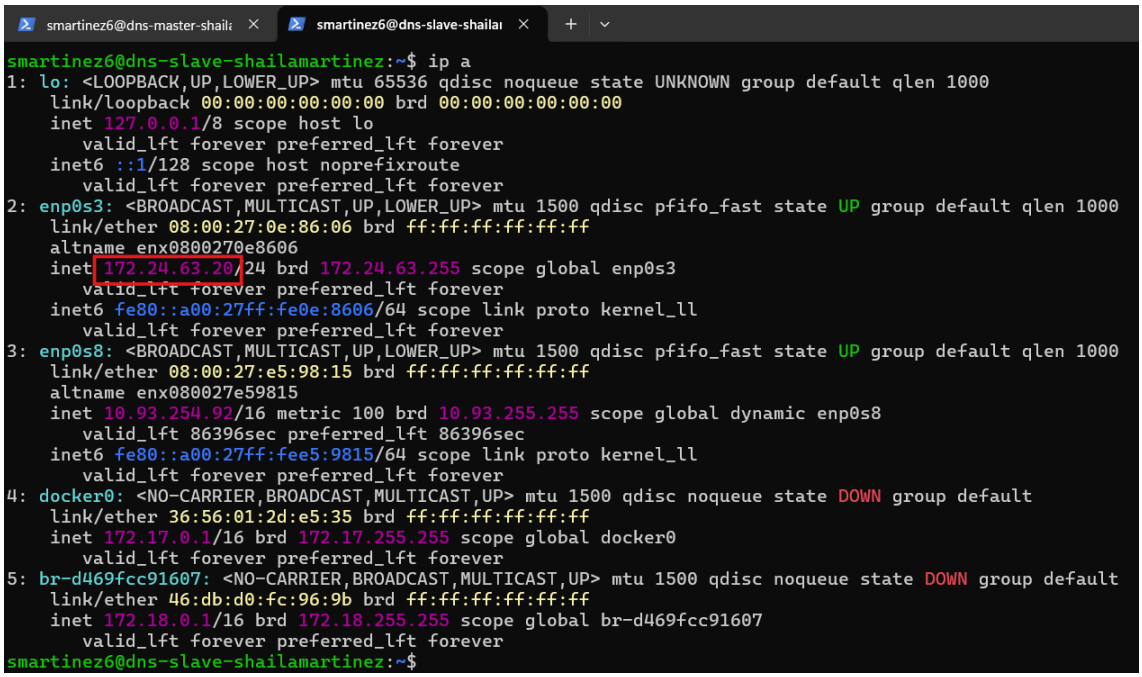


Des del client he comprovat que els subdominis resolen correctament.

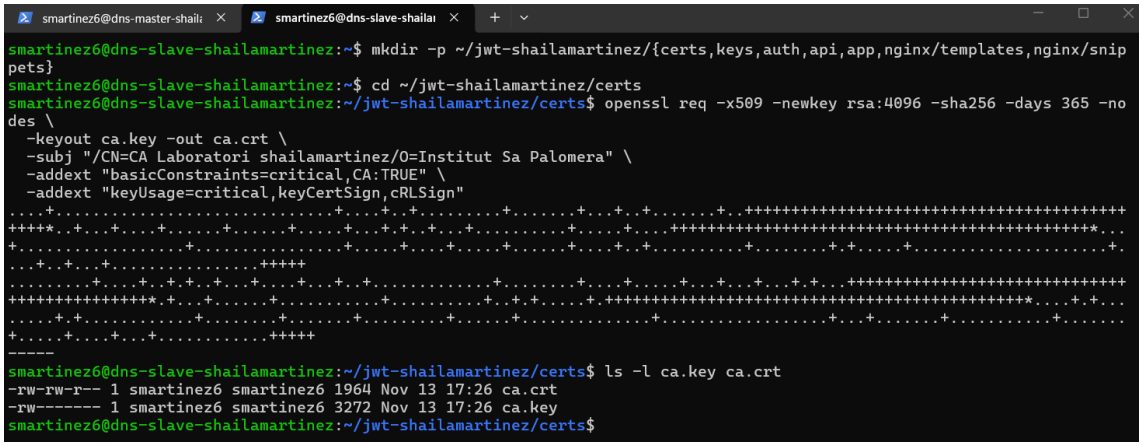


Part 2: CA pròpia i certificat wildcard

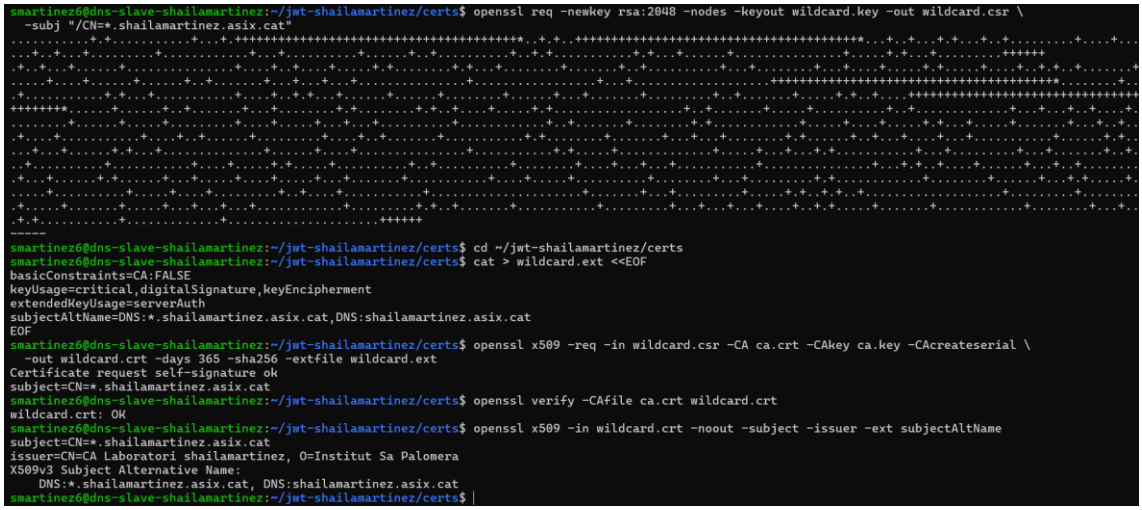
IP slave:



Creo un certificat propi amb OpenSSL. Servirà per signar el certificat que utilitzaré als subdominis. He configurat el certificat com a CA i li he donat permisos per signar altres certificats, s'han generat els fitxers ca.crt i ca.key.



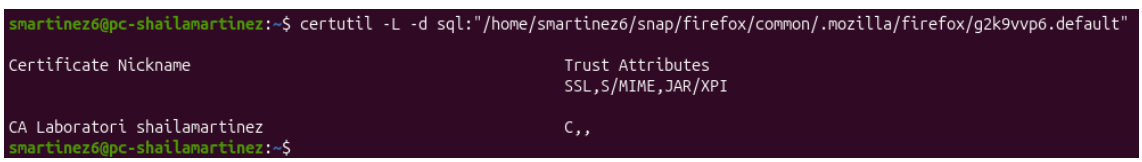
Certificat Wildcard per utilitzar el mateix certificat als subdominis auth, api i app sense crear un diferent per a cadascun. He verificat que el certificat està signat amb la meva CA.



He copiat el certificar públic de CA al client i l'he instal·lat amb update-ca-certificates.



Després he comprovat que també apareix a la base de certificats del Firefox.

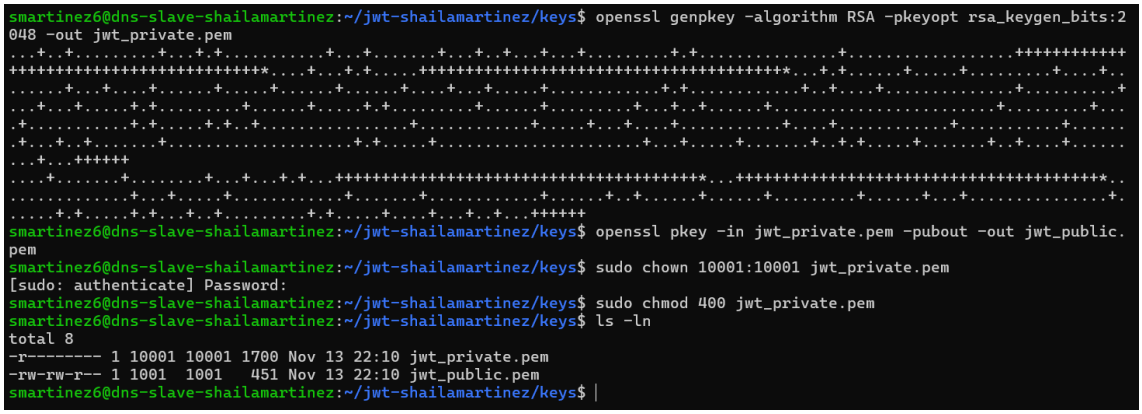


Part 3: Claus JWT i serveis d'autenticació i API

3.1 Parell de claus RS256

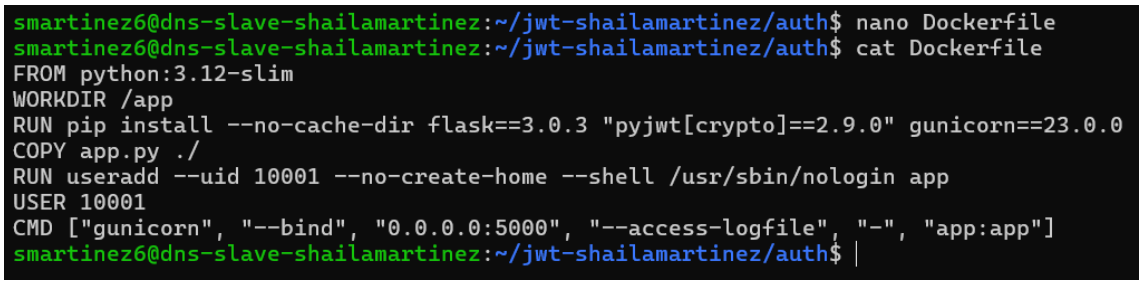
He generat un parell de claus RSA per poder treballar amb RS256. La clau privada (jwt_private.pem) és la que utilitza el servei auth per signar els tokens, mentre que la clau pública (jwt_public.pem) serveix per comprovar que els tokens estan ben signats.

He canviat el propietari de la clau privada a l'usuari amb uid 10001 i li he posat permisos 400. Així només la pot llegir l'usuari que executa el servei i evito que altres processos puguin utilitzar-la.



3.2 Servei auth

auth/Dockerfile: és que gestiona l'inici de sessió i genera els JWT. Utilitza python i gunicorn per executar Flask.



auth/app.py: carrega les claus, dades dels usuaris i configuració del domini. Prepara el formulari de login i les funcions per generar i validar els tokens.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/auth$ sudo nano app.py
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/auth$ cat app.py
"""Servei d'autenticació: emet i valida JWT signats amb RS256."""
import json
import os
import time
import uuid

import jwt
from flask import Flask, jsonify, make_response, redirect, request
from werkzeug.security import check_password_hash

DOMINI = os.environ["DOMINI"]                # p. ex. shailamartinez.asix.cat
ISSUER = f"https://auth.{DOMINI}"
AUDIENCE = f"api.{DOMINI}"
TTL = int(os.environ.get("TOKEN_TTL", "900"))  # segons (15 min)

with open("/keys/jwt_private.pem") as f:
    PRIVATE_KEY = f.read()
with open("/keys/jwt_public.pem") as f:
    PUBLIC_KEY = f.read()
with open("/app/users.json") as f:
    USERS = json.load(f)

app = Flask(__name__)
app.json.ensure_ascii = False  # accents llegibles a les respostes JSON

FORMULARI = f"""<!doctype html>
<html lang="ca"><head><meta charset="utf-8"><title>Login · {DOMINI}</title></head>
<body>
  <h1>Inici de sessió — {DOMINI}</h1>
  <form method="post" action="/login">
    <label>Usuari <input name="username" autocomplete="username"></label><br>
    <label>Contrasenya <input name="password" type="password" autocomplete="current-password"></label><br
    <button type="submit">Entra</button>
```

3.3 Servei api

Creo Dockerfile amb la configuració semblant a la del auth. Utilitza python, flask, gunicorn i s'executa amb un usuari sense privilegis.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/auth$ cd ~/jwt-shailamartinez/api
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$ nano Dockerfile
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$ cat Dockerfile
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir flask==3.0.3 "pyjwt[crypto]==2.9.0" gunicorn==23.0.0
COPY app.py ./
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
USER 10001
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--access-logfile", "-", "app:app"]
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$
```

Codi de l'Api, carrega la clau pública perquè ha de comprovar els JWT que rep i no generar nous.

/api/public: és públic i no necessita cap token

/api/perfil: necessita un token vàlid i mostra les dades de l'usuari

/api/admin: només permet l'accés als usuaris que són admin

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$ cat app.py
"""API protegida: només té la clau PÚBLICA, pot validar tokens però no emetre'n."""
import os

import jwt
from flask import Flask, g, jsonify, request

DOMINI = os.environ["DOMINI"]
ISSUER = f"https://auth.{DOMINI}"
AUDIENCE = f"api.{DOMINI}"

with open("/keys/jwt_public.pem") as f:
    PUBLIC_KEY = f.read()

app = Flask(__name__)
app.json.ensure_ascii = False  # accents llegibles a les respostes JSON


def error(codi, missatge):
    resposta = jsonify(error=missatge)
    resposta.status_code = codi
    if codi == 401:
        resposta.headers["WWW-Authenticate"] = 'Bearer realm="api"'
    return resposta


@app.before_request
def autentica():
    if request.path == "/api/public":
        return None
    capcalera = request.headers.get("Authorization", "")
    token = capcalera.removeprefix("Bearer ") if capcalera.startswith("Bearer ") else request.cookies.get("access_token")
    if not token:
        return error(401, "falta el token")
    try:
        g.claims = jwt.decode(
            token, PUBLIC_KEY, algorithms=["RS256"], audience=AUDIENCE, issuer=ISSUER,
            options={"require": ["exp", "iat", "iss", "aud", "sub"]},
        )
    except jwt.ExpiredSignatureError:
        return error(401, "token caducat")
    except jwt.InvalidTokenError as e:
        return error(401, f"token invàlid: {e}")
    return None


@app.get("/api/public")
def public():
    return jsonify(missatge=f"API de {DOMINI}: endpoint públic")


@app.get("/api/perfil")
def perfil():
    return jsonify(usuari=g.claims["sub"], rol=g.claims["role"], caduca=g.claims["exp"])


@app.get("/api/admin")
def admin():
    if g.claims.get("role") != "admin":
        return error(403, "cal el rol admin")
    return jsonify(missatge=f"Benvingut/da a l'administració, {g.claims['sub']}")
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$
```

3.4 Pàgina privada

Pàgina HTML per a la zona privada. Només per veure quan l'usuari ha iniciat sessió i nginx ha comprovat que el seu JWT és vàlid, utilitzant auth_request. També per tancar sessió.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/api$ cd ~/jwt-shailamartinez/app
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/app$ nano index.html
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/app$ cat index.html
<!doctype html>
<html lang="ca"><head><meta charset="utf-8"><title>Zona privada</title></head>
<body>
  <h1>Zona privada de shailamartinez.asix.cat</h1>
  <p>Si veus aquesta pàgina, Nginx ha validat el teu JWT amb <code>auth_request</code>.</p>
  <form method="post" action="https://auth.shailamartinez.asix.cat/logout"><button>Tanca la sessió</button></form>
</body></html>
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/app$
```

3.5 Docker Compose i usuaris

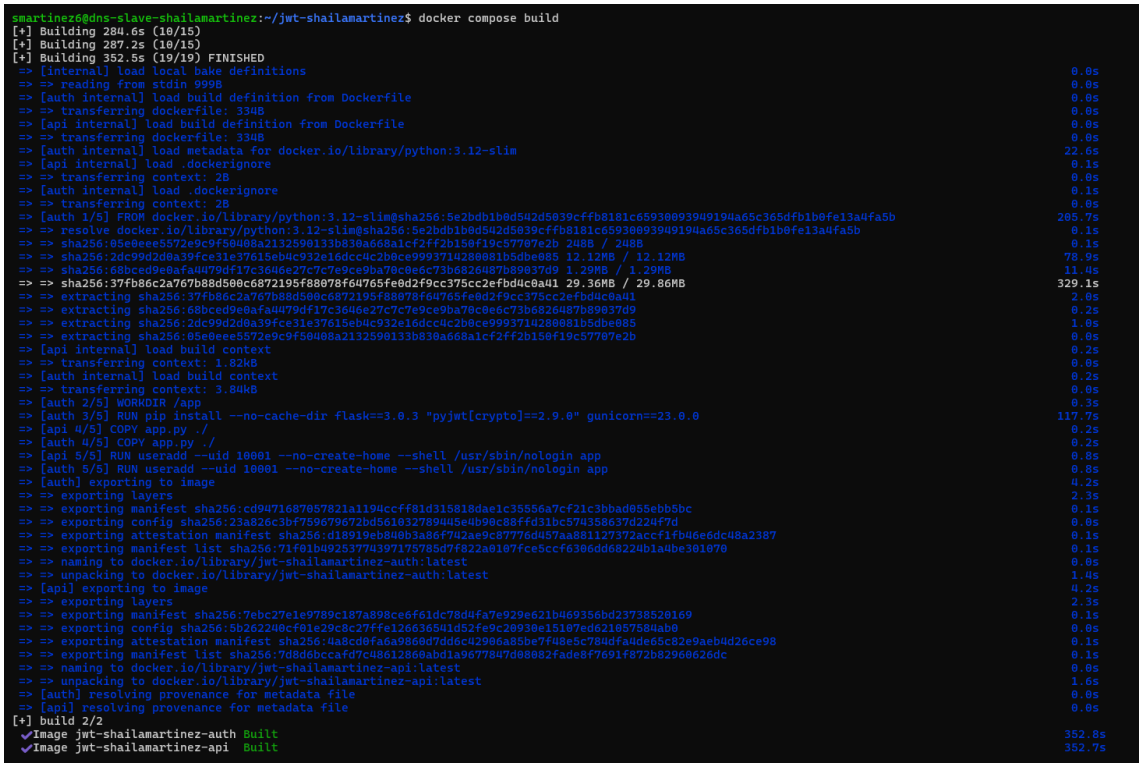
.env: amb el domini i durada inicial del token de 900 segons. A docker-compose.yml s'han definit nginx, auth i api, també els volums perquè da contenidor tingui accés només als fitxers que necessita.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez/app$ cd ~/jwt-shailamartinez
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ nano .env
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat .env
DOMINI=shailamartinez.asix.cat
TOKEN_TTL=900
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ nano docker-compose.yml
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat docker-compose.yml
services:
  nginx:
    image: nginx:1.27-alpine
    container_name: nginx-jwt
    environment:
      DOMINI: ${DOMINI}
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/templates:/etc/nginx/templates:ro
      - ./nginx/snippets:/etc/nginx/snippets:ro
      - ./app:/usr/share/nginx/html:ro
      - ./certs/wildcard.crt:/etc/nginx/certs/wildcard.crt:ro
      - ./certs/wildcard.key:/etc/nginx/certs/wildcard.key:ro
    depends_on:
      - auth
      - api
    restart: unless-stopped

  auth:
    build: ./auth
    container_name: auth-jwt
    environment:
      DOMINI: ${DOMINI}
      TOKEN_TTL: ${TOKEN_TTL}
    volumes:
      - ./keys/jwt_private.pem:/keys/jwt_private.pem:ro
      - ./keys/jwt_public.pem:/keys/jwt_public.pem:ro
      - ./auth/users.json:/app/users.json:ro
    restart: unless-stopped

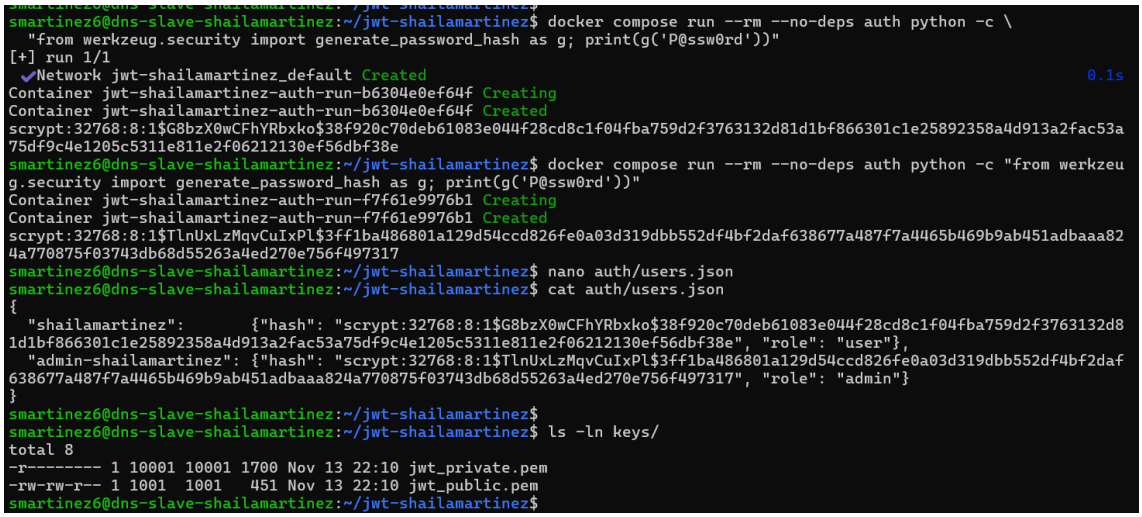
  api:
    build: ./api
    container_name: api-jwt
    environment:
      DOMINI: ${DOMINI}
    volumes:
      # L'API NOMÉS rep la clau pública
      - ./keys/jwt_public.pem:/keys/jwt_public.pem:ro
    restart: unless-stopped
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$
```

He construït les imatges:



Generació dels hash de les contrasenyes el mateix contenidor auth. Després he creat dos usuaris al fitxer users.json: un amb el rol user i un altre amb el rol admin.

Comprovació dels permisos de les claus:



Part 4: Nginx — tres llocs virtuals, TLS i auth_request

Creació del fitxer seguretat.conf amb la configuració TLS i capçaleres de seguretat que s'aplicaran als llocs. Protegeixen la connexió.

Proxy.conf amb les capçaleres que nginx envia, com la IP real del client i el protocol.

```
smartinez6@dns-master-shaila  ×  smartinez6@dns-slave-shailar  ×  +  ⌄
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ nano nginx/snippets/seguretat.conf
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat nginx/snippets/seguretat.conf
ssl_certificate     /etc/nginx/certs/wildcard.crt;
ssl_certificate_key /etc/nginx/certs/wildcard.key;
ssl_protocols       TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;

add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options    "nosniff" always;
add_header X-Frame-Options           "DENY" always;
add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy   "default-src 'self'; object-src 'none'; frame-ancestors 'none'" always;
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ nano nginx/snippets/proxy.conf
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat nginx/snippets/proxy.conf
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$
```

També he començat a configurar jwt.conf.template, on es defineixen els diferents llocs.

auth: gestiona el formulari de login i l'emissió dels tokens.

api: envia les peticions a l'API, que és qui valida els JWT.

app: serveix la pàgina privada i utilitza auth_request per comprovar el token abans de mostrar-la.

Quan supera el límit de peticions al login, nginx retorna error 429.

```
smartinez6@dns-master-shaila  ×  smartinez6@dns-slave-shailar  ×  +  ⌄
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ nano nginx/templates/jwt.conf.template
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat nginx/templates/jwt.conf.template
# Les variables ${DOMINI} les substitueix la imatge oficial de Nginx en arrencar
# (fitxers *.template de /etc/nginx/templates → /etc/nginx/conf.d)

server_tokens off;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

# HTTP → HTTPS per a tots els subdominis
server {
    listen 80;
    server_name .${DOMINI};
    return 301 https://$host$request_uri;
}

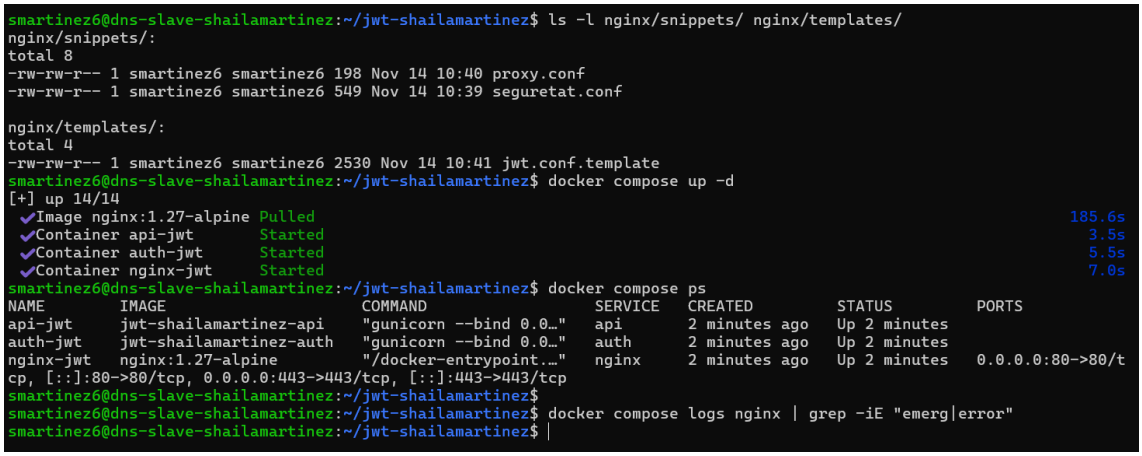
# Peticions que no són per a cap dels nostres noms (escanejos per IP)
server {
    listen 443 ssl default_server;
    server_name _;
    include /etc/nginx/snippets/seguretat.conf;
    return 444;
}

# ---------- auth.DOMINI: emissió de tokens ----------
server {
    listen 443 ssl;
    http2 on;
    server_name auth.${DOMINI};
    include /etc/nginx/snippets/seguretat.conf;

    location = /login {
        limit_req zone=login burst=3 nodelay;
        limit_req_status 429;
        proxy_pass http://auth:5000;
        include /etc/nginx/snippets/proxy.conf;
    }

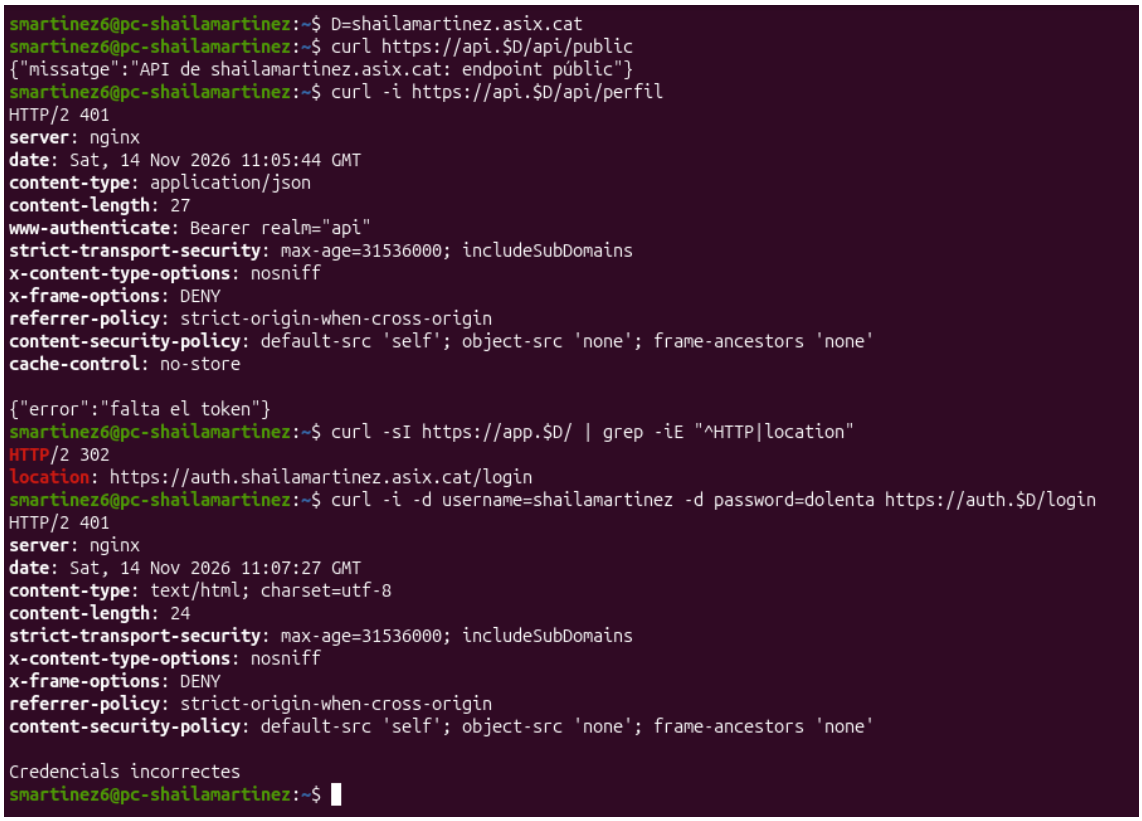
    # /verify és només per a ús intern de Nginx
    location = /verify {
        return 404;
```

He iniciat tots els serveis amb i després he comprovat l'estat. Els logs no mostren errors:



Part 5: Proves funcionals

Comprovació que /api/públic funciona sense necessitat d'autenticar-se. He provat /api/perfil sense enviar cap token i 'api ha retornat un error 401 inicant que falta el token. També comprovo que si s'intenta accedir aapp.shailamartinez.asix.cat sense have iniciat sessió, nginx redirigeix al formulari login. He provat d'iniciar sessió amb una contrasenya incorrecta i el sistema ha retornat un error 401 de credencials incorrectes.



He iniciat sessió amb l'usuari normal i he guardat el JWT que m'ha retornat el servei auth. Després he utilitzat aquest token per accedir a /api/perfil. El resultat mostra correctament l'usuari shailamartinez i el seu rol user. Quan he intentat entrar a /api/admin amb aquest mateix token, l'Api m'ha retornat un error 403, perquè l'usuari no té permisos d'administrador.

El mateix token també permet accedir a la privada per la cookie access_token. A les proves del login he fet diversos intents i es pot veure que després de superar el límit, Nginx retorna errors 429.

Accés per IP Nginx tanca la connexió:

```
smartinez6@pc-shailamartinez:~$ TOKEN=$(curl -s -H 'Accept: application/json' \
        -d username=shailamartinez -d 'password=P@ssw0rd' \
        https://auth.$D/login | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
smartinez6@pc-shailamartinez:~$ echo "$TOKEN"
eyJhbGciOiJSUzI1NiIsImtpZCI6ImNsYXUtMSIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2F1dGguc2hhaWxhbWFydGluZXouYXNpeC5jYXQiLCJhdWQiOiJhcGkuc2hhaWxhbWFydGluZXouYXNpeC5jYXQiLCJzdWIiOiJzaGFpbGFtYXJ0aW5leiIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzk0NjU0NTM3LCJleHAiOjE3OTQ2NTU0MzcsImp0aSI6ImRjMTg4YzMzLTE5NGMtNDdkMS1iOWZjLWExNjY1MGM0OTM3NSJ9.j0kXtxup6SyQT9wQDJdNHdc_YeKpufxO6rv5oO3oGJRC4H4LJal90Flr8aePT2Y_6giekq0sjGxFWoSQFLkCe2hjLCLTAEyivOpWA1LCqK5c6uCFyOkvth1uW_nRxq2zZtVxQ_2wvRpNtAOtTkDktD30D-Fx1JHYi4aq8CAk6IwsW1aBp3W0HHRxisjHTjeRh8ccuwkhxeZbK3Z9Is_N0sBDKc_Fn9AYBfLRbGEOkJYNY_wM5UDAzUQTQ0ybCCRUUG7sBkdi3VcWWst783ZB1Be1pUaTHuzU9Rsm2aQzrHB8Gkd0laMAurb9eFhpH6KSMaqmBE_m4x62dfDNi1Z7vA
smartinez6@pc-shailamartinez:~$ ^C
smartinez6@pc-shailamartinez:~$ curl -H "Authorization: Bearer $TOKEN" https://api.$D/api/perfil
{"caduca":1794655437,"rol":"user","usuari":"shailamartinez"}
smartinez6@pc-shailamartinez:~$ curl -i -H "Authorization: Bearer $TOKEN" https://api.$D/api/admin
HTTP/2 403
server: nginx
date: Sat, 14 Nov 2026 11:10:46 GMT
content-type: application/json
content-length: 29
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'self'; object-src 'none'; frame-ancestors 'none'
cache-control: no-store

{"error":"cal el rol admin"}
smartinez6@pc-shailamartinez:~$ curl -sI -b "access_token=$TOKEN" https://app.$D/ | grep -iE "^HTTP|x-usuari|x-rol"
HTTP/2 200
x-usuari: shailamartinez
x-rol: user
smartinez6@pc-shailamartinez:~$ for i in $(seq 1 6); do curl -s -o /dev/null -w '%{http_code} ' -d username=x -d password=x https://auth.$D/login; don
e; echo
401 401 401 401 429 429
smartinez6@pc-shailamartinez:~$ curl -k https://172.24.63.20/
curl: (56) OpenSSL SSL_read: OpenSSL/3.5.5: error:0A000126:SSL routines::unexpected eof while reading, errno 0
smartinez6@pc-shailamartinez:~$
```

He repetit el procés d'inici de sessió utilitzant l'usuari admin-shailamartinez, com que té rol admin, el resultat és correcte i l'api dona la benvinguda.

```
smartinez6@pc-shailamartinez:~$ ADMIN_TOKEN=$(curl -s -H 'Accept: application/json' \
  -d username=admin-shailamartinez -d 'password=P@ssw0rd' \
  https://auth.$D/login | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
smartinez6@pc-shailamartinez:~$ echo "$ADMIN_TOKEN"
eyJhbGciOiJSUzI1NiIsImtpZCI6ImNsYXUtMSIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2F1dGguc2hhaWxhbWFydGluZXouYXNpeC5jYXQiLCJhdWQiOiJhcGkuc2hhaWxhbWFydGluZXouYXNpeC5jYXQiLCJzdWIiOiJhZG1pbi1zaGFpbGFtYXJ0aWSleiIsInJvbGUiOiJhZG1pbiIsImlhdCI6MTc5NDY1NDkyMywiZXhwIjoxNzk0NjU1ODIzLCJqdGkiOiI2YmRhNjk0NC04YWY1LTRmOWQtOThjMS05MmM0Mzg1YjU5MjAifQ.cr9p0w-3DDvmL6y2BMkOP-9ZlsX7PQDlzEDpSpXY4ZLKjayaaxdIuwCESLdUzSG0WMlClpbAc0Ch6_-B5OVCVbXu2enl2hDL2diZWXm_r0-x26MUk0-TYqv4ii9j67hzJKUYBukENN5NGjLvX0aCGA0OpRORxcGYw1l2BByv9bvszXfQvUrLc2f0T0Q7VsxBmeI3v4GPs39a1KZy3duLTCHAFkp0-tPHC9b2BYS8lqu6tIwe1oZSMBycbriVCINsNHyU7h69arcK5S5dfjtm3PN2vcFrPfhU_9eff4ZKpAUjpugGfrCGCzui5N5e2PfImUXihLBDtigUj-IWizbDVQ
smartinez6@pc-shailamartinez:~$ curl -i -H "Authorization: Bearer $ADMIN_TOKEN" https://api.$D/api/admin
HTTP/2 200
server: nginx
date: Sat, 14 Nov 2026 11:16:11 GMT
content-type: application/json
content-length: 69
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'self'; object-src 'none'; frame-ancestors 'none'
cache-control: no-store

{"missatge":"Benvingut/da a l'administració, admin-shailamartinez"}
smartinez6@pc-shailamartinez:~$
```

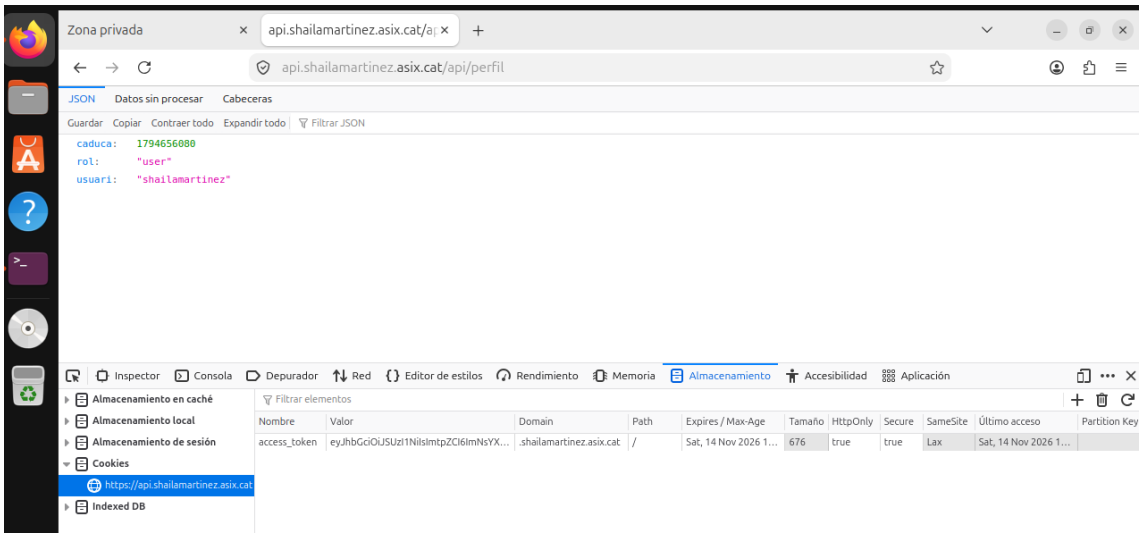
He obert app.shailamartinez.asix.cat amb Firefox. Com que encara no havia iniciat sessió, m'ha redirigit automàticament al formulari del subdomini auth.



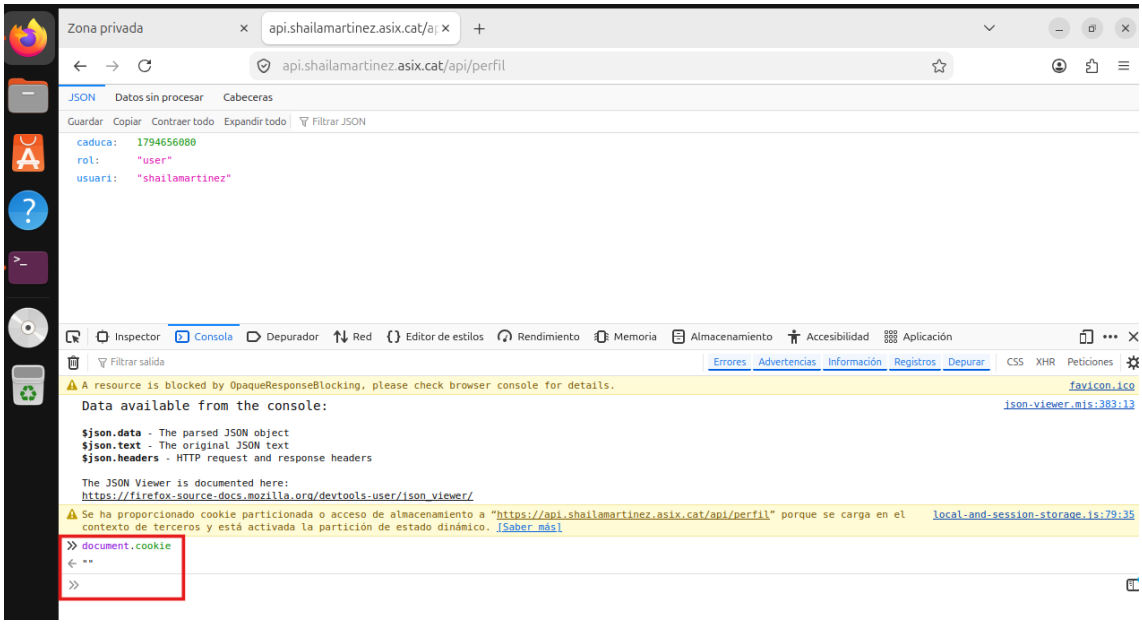
Després d'iniciar sessió, redirigeix a app.shailamartinez.asix.cat i es pot veure la pàgina privada sense haver de tornar a introduir les credencials.



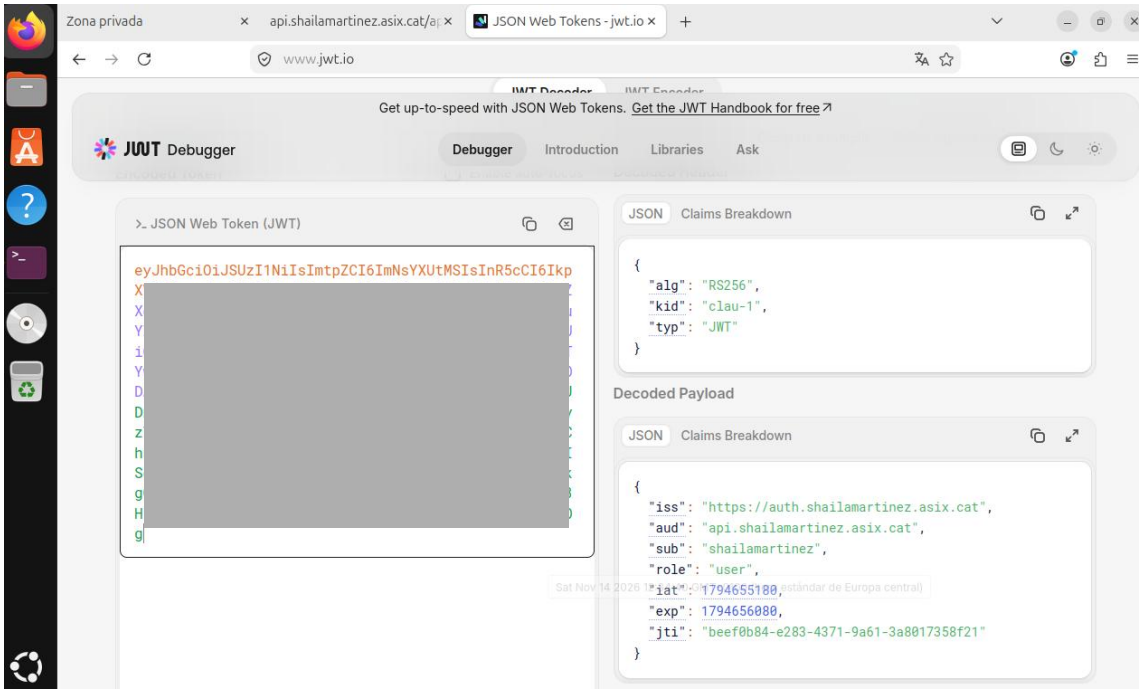
api: mostra les dades de l'usuari, confirma que el navegador també envia la cookie a l'api. Atributs de la cookie access_token:



La comanda document.cookie retorna buit “” perquè la cookie està configurada amb HttpOnly. No es pot llegir directament JWT.



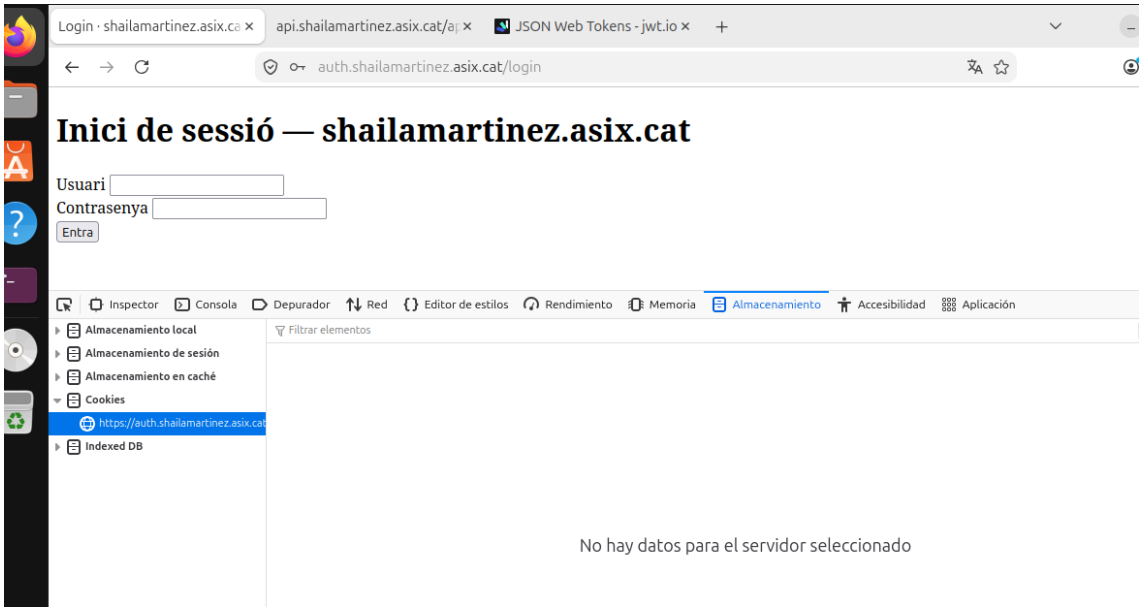
He t el token a jwt.io per poder veure el seu contingut i comprovar les claims. El token utilitza l'algorisme RS256 i conté informació com iss, aud... Permet comprovar que el token conté la informació que necessita per autenticar l’usuari.



- iss: qui ha emès el token
- aud: a qui va destinat el token (l’api)
- sub: l’usuari propietari del token
- role: el rol de l’usuari (user o admin)
- exp: quan caduca el token

- jti: identificador únic del token

Tanco la sessió i comprovo que la cookie access_token ja no apareix al navegador. L'usuari ha de tornar a iniciar sessió per accedir a la zona privada.



Part 6: Proves d'atac contra el JWT

Atac 1: He intentat modificar el payload del JWT per canviar el rol de user a admin, he enviat el token modificat a l'api. El server l'ha denegat amb error 401 perquè la signatura ja no coincideix amb el contingut del token.

Missatge d'error de l'api: token invàlid: Signature verification failed. Ho atura la línia jwt.decode(token, PUBLIC_KEY, algorithms=...) de api/app.py

Atac 2: s'ha intentat crear un token amb alg:none (token sense signatura) i l'ha denegat perquè el servidor només accepta tokens signats amb RS256 i no confia en els que indica el mateix token.

token invàlid: The specified alg value is not allowed. Ho atura el paràmetre algorithms=["RS256"] del jwt.decode, la llista blanca d'algorismes.

```
smartinez6@pc-shailamartinez:~$ b64url() { openssl base64 -e -A | tr '+/' '-_' | tr -d '='; }
b64url_decode() { local s; s=$(tr '_-' '/+' <<<"$1"); while (( ${#s} % 4 )); do s+='='; done; base64 -d <<<"$s"; }

IFS=. read -r H P SIG <<<"$TOKEN"
prova() { echo "--- $1"; curl -s -w ' [%{http_code}]\n' -H "Authorization: Bearer $2" https://api.$D/api/admin; }
smartinez6@pc-shailamartinez:~$ P_ADMIN=$(b64url_decode "$P" | sed 's/"role":"user"/"role":"admin"/' | b64url)
smartinez6@pc-shailamartinez:~$ prova "payload manipulat" "$H.$P_ADMIN.$SIG"
--- payload manipulat
{"error":"token invàlid: Signature verification failed"}
 [401]
smartinez6@pc-shailamartinez:~$ H_NONE=$(printf '{"alg":"none","typ":"JWT"}' | b64url)
smartinez6@pc-shailamartinez:~$ prova "alg none" "$H_NONE.$P_ADMIN."
--- alg none
{"error":"token invàlid: The specified alg value is not allowed"}
 [401]
smartinez6@pc-shailamartinez:~$
```

Atac 3: El servidor ha rebutjat el token amb un error 401, perquè només permet l'algorisme RS256.

token invàlid: The specified alg value is not allowed. Es denega pel mateix motiu que l'atac 2, HS256 no és a la llista blanca algorithms=RS256, així que no s'accepta que la clau pública es faci servir com a secret HMAC.

Atac 4: He generat un altre parell de claus RSA i he utilitzat la clau privada de l'atacant per signar un token manipulat. Retorna error 401 perquè la signatura no correspon a la clau pública que té configurada el server.

token invàlid: Signature verification failed. Ho atura la mateixa línia jwt.decode(token, PUBLIC_KEY, algorithms= ...): la signatura no es pot verificar amb la clau pública del servidor.

```
smartinez6@pc-shailamartinez:~$ mkdir -p ~/keys
smartinez6@pc-shailamartinez:~$ scp smartinez6@172.24.63.20:~/jwt-shailamartinez/keys/jwt_public.pem ~/keys/
smartinez6@172.24.63.20's password:
jwt_public.pem                                                                                100%  451    32.0KB/s   00:00
smartinez6@pc-shailamartinez:~$ H_HS=$(printf '{"alg":"HS256","typ":"JWT"}' | b64url)
SIG_HS=$(printf '%s.%s' "$H_HS" "$P_ADMIN" | openssl dgst -sha256 -hmac "$(cat keys/jwt_public.pem)" -binary | b64url)
smartinez6@pc-shailamartinez:~$ prova "confusio d'algorisme" "$H_HS.$P_ADMIN.$SIG_HS"
--- confusio d'algorisme
{"error":"token invàlid: The specified alg value is not allowed"}
 [401]
smartinez6@pc-shailamartinez:~$
smartinez6@pc-shailamartinez:~$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out /tmp/atacant.pem
.......+.++++++++++++++++++++++++++++++++++++++++*....+.+.....+...+...+.....+......+...+.+.....+.+.....+.....+.....+.....+.....+...+...+..+
++++++++++++++++++++++++++++++++++++++++*.+....+..........+.........+.......+...........+.....+......+.+.....................+.............+...+...+..+
.........+....+...........+...+...+...+.......+..+.....................+..........+...+.+...+.................+.+............+..+...+..........+..
....+.............+..+.......+..+.+.........+....+...+.+.....+.............+....+...+...+.............+.....+....+...+..+.........+.........+.....
.....+...+......+.......+..+.+.+.....+...+............+.....+.............+..+.........+...+.+....+..+....+........+.+..+...+....+...+..+.+.........
.+..+..............+.............+.............+.............+.....+.....+.......+..+...+..+.+.....+.....+..+.+..+.....+.....+........+...+..
.......+.........+.........+...+....+...+......+.....+.........+.+.........+..+.........+......+..............+..+.................+..........+++
+++
....++++++++++++++++++++++++++++++++++++++++*.....+.....+.+...+......+.....+.+........+.+..........+....+.....+....+..+......++++++++++++++++++++++++
++++++++++++++*.+.....+......+......+.+........+.+..................+......++++++
smartinez6@pc-shailamartinez:~$ H_RS=$(printf '{"alg":"RS256","typ":"JWT"}' | b64url)
SIG_RS=$(printf '%s.%s' "$H_RS" "$P_ADMIN" | openssl dgst -sha256 -sign /tmp/atacant.pem -binary | b64url)
smartinez6@pc-shailamartinez:~$ prova "clau aliena" "$H_RS.$P_ADMIN.$SIG_RS"
--- clau aliena
{"error":"token invàlid: Signature verification failed"}
 [401]
smartinez6@pc-shailamartinez:~$
```

Atac 5: He canviat el valor del token ttl de 900 a 30 segons al fitxer .env. He reiniciat el servei auth i nginx.

token caducat. Ho atura el bloc except jwt.ExpiredSignatureError de api/app.py, que salta quan la claim exp ja ha passat.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ sudo nano .env
[sudo: authenticate] Password:
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat .env
DOMINI=shailamartinez.asix.cat
TOKEN_TTL=30
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ docker compose up -d auth && docker compose restart nginx
[+] up 1/1
 ✔ Container auth-jwt Started                                                                    3.7s
[+] restart 0/1
 ⠋ Container nginx-jwt Restarting                                                                3.9s
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$
```

He generat un token nou i esperat perquè caduqués. Quan he intentat utilitzar-lo a /api/perfil, l'api denega indicant que el token està caducat.

```
smartinez6@pc-shailamartinez:~$ TOKEN=$(curl -s -H 'Accept: application/json' \
  -d username=shailamartinez -d 'password=P@ssw0rd' \
  https://auth.$D/login | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
smartinez6@pc-shailamartinez:~$ sleep 40
smartinez6@pc-shailamartinez:~$ curl -i -H "Authorization: Bearer $TOKEN" https://api.$D/api/perfil
HTTP/2 401
server: nginx
date: Sat, 14 Nov 2026 12:07:03 GMT
content-type: application/json
content-length: 26
www-authenticate: Bearer realm="api"
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'self'; object-src 'none'; frame-ancestors 'none'
cache-control: no-store

{"error":"token caducat"}
smartinez6@pc-shailamartinez:~$ curl -sI -b "access_token=$TOKEN" https://app.$D/ | grep -iE "^HTTP|location"
HTTP/2 302
location: https://auth.shailamartinez.asix.cat/login
smartinez6@pc-shailamartinez:~$
```

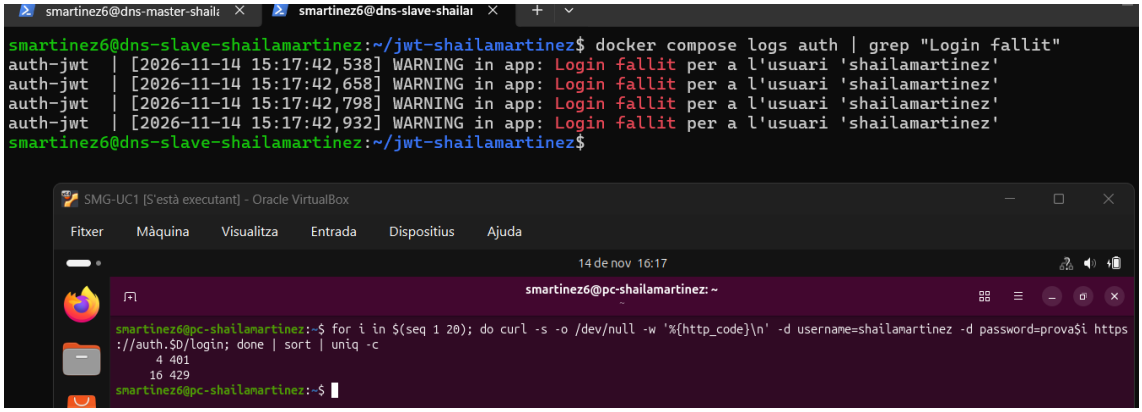
Torno a posar 900 segons.

```
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ cat .env
DOMINI=shailamartinez.asix.cat
TOKEN_TTL=900
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$ docker compose up -d auth && docker compose restart nginx
[+] up 1/1
 ✔ Container auth-jwt Started                                                                    3.3s
[+] restart 0/1
 ⠋ Container nginx-jwt Restarting                                                                1.9s
smartinez6@dns-slave-shailamartinez:~/jwt-shailamartinez$
```

Atac 6: s'han fet 20 intents seguits de login amb contrasenyes incorrectes, el resultat ha estat 4 respostes 401 i 16 respostes 429. Nginx ha deixat passar els primers intents, però després ha bloquejat la majoria de peticions perquè ha superat el límit configurat.

He consultat els logs del servei auth on apareixen es 4 intents que han arribat.

Ho atura el rate limiting de Nginx (limit_req_zone ... rate=5r/m i limit_req zone=login burst=3 nodelay a location = /login), els intents que superen el límit reben un 429 i només 4 arriben a auth.



Qüestions

1. Per què el servei api només rep jwt_public.pem? Què podria fer un atacant que comprometés el contenidor api si també tingués la clau privada?

Perquè la api només necessita la clau pública per comprovar que els JWT estan ben signats, la clau privada només ha d'estar al servei auth que és el que genera i signa els tokens. Si un atacant aconsegueix la privada des de la api, podria crear els seus propis tokens i posar-se com administrador i accedir als serveis, es faria passar per qualsevol usuari.

2. Expliqueu la diferència entre els dos models de validació de la pràctica (l'aplicació a api. i Nginx amb auth_request a app.). Quin avantatge té cada un?

api. és la mateixa aplicació qui comprova el JWT que rep Authorization abans de permetre l'accés a cada endpoint, això permet controlar els permisos segons el rol de l'usuari.

En app. és Nginx qui comprova el token mitjançant auth_request abans de mostrar la pàgina privada. L'avantatge és que no fa falta programar aquesta comprovació dins de la pàgina web perquè nginx controla l'accés directament.

3. La cookie porta Domain=shailamartinez.asix.cat per poder fer SSO. A Autenticació web i JWT es recomana el prefix __Host-. Per què no el podem fer servir aquí? Quin risc acceptem i com el mitigaríeu?

Perquè obliga a no posar l'atribut Domain i es necessita compartir la cookie entre els subdominis auth, app i api per poder tenir el SSO.

El risc és que la cookie també s'envia als altres subdominis i si algun d'ells està compromès, podria posar en perill la sessió. Utilitzaria Secure Httponly i SameSite

= Lax per mantenir els subdominis protegits amb https i el token tingui una durada limitada.

4. Un JWT emès té validesa fins a l'exp. Si es roba un token, com el podríeu invalidar abans? Feu servir la claim jti a la resposta.

Es podria crear una llista de tokens revocats al servidor, cada JWT té un claim jti que és un id únic. Si es roba un token, podem guardar el seu jti en aquesta llista i comprovar-lo cada vegada que es valida un JWT. Si apareix a la llista, es denega encara que no hagi arribat a la data de caducitat.

5. Per què el login retorna 401 tant si l'usuari no existeix com si la contrasenya és incorrecta, i no dos missatges diferents?

Per evitar que un atacant pugui saber quins usuaris existeixen, si es mostren missatges diferents es podria provar noms d'usuari i descobrir quins són vàlids.

6. Què passaria amb el SSO si auth. fos a shailamartinez.asix.cat i app. a un altre domini (per exemple botiga.cat)? Quin protocol es fa servir a la vida real per resoldre-ho?

Com pertany a un domini diferent ja no es podria compartir i l'usuari no mantindria la sessió automàticament entre els dos dominis.

Es pot resoldre OpenId connect, l'usuari inicia sessió en un servei central i després les altres aplicacions poden reconèixer que ja s'ha identificat, sense haver de compartir la mateixa cookie entre elles.